專題進度 1 報告(學期中)

專題名稱：視覺化網路攻擊自動化分析平台

成員：D1378300 胡安硯、D1300043 何昇駿、D1256832 石任立

一、工作與進度報告：

目前的程式架構圖：

```
network_capture/
│
├── config.py                 全域設定中心（閾值、路徑、協議對應表）
├── parser.py                 封包欄位解析（從原始封包提取 IP、Port、Flags 等）
├── anomaly_detector.py       攻擊偵測引擎（SYN Flood / Port Scan / ARP Spoof 等）
├── capture.py                即時封包擷取（監聽網路卡）
├── pcap_analyzer.py          離線 PCAP 分析（讀取已存的封包檔案）
├── packet_visualizer.py      封包影像化核心（bytes -> 32x32 灰階矩陣）
├── dataset_builder.py        訓練資料集建構（批次轉換 + 儲存 .npy）
├── dataset_loader.py         真實資料集載入（NSL-KDD / CIC-IDS2017 / CIC-DDoS2019）
├── cnn_autoencoder.py        CNN Autoencoder 模型定義（Encoder + Decoder）
├── trainer.py                模型訓練器（Early Stopping / 閾值計算）
├── anomaly_scorer.py         異常分數計算與效能評估（ROC / Precision / Recall）
├── run_training.py           端對端訓練腳本（一鍵執行完整流程）
├── storage.py                資料儲存（CSV / JSON / SQLite）
├── session_manager.py        Session 管理（每次測試建立獨立紀錄目錄）
├── cleaner.py                輸出結果清空工具（清除 sessions / dataset / model 等）
├── main.py                   命令列介面入口（live / pcap / session / clean）
│
├── tests/
│   ├── test_parser.py            解析器單元測試
│   ├── test_anomaly.py           攻擊偵測單元測試
│   ├── test_visualization.py     影像化功能測試
│   ├── test_cnn_model.py         CNN 模型單元測試
│   ├── test_dataset_loader.py    資料集載入測試
│   └── generate_attack_pcap.py   攻擊模擬封包生成器（PCAP）
│
└── output/
    ├── sessions/                 每次擷取的 Session 紀錄
    ├── dataset*/                 封包影像矩陣（.npy 格式）
    ├── model*/                   CNN 模型與訓練圖表
    └── visualization_demo/       影像化展示圖（PNG）
```

網路封包擷取與解析：

封包解析的核心是 PacketParser，負責將 Scapy 讀入的原始封包物件逐層拆解，從 Ethernet、IP 到 TCP/UDP/ICMP，再解析到 DNS 查詢與 TLS 握手類型，輸出一個包含所有欄位的標準化字典，作為系統所有上層功能的統一資料來源。在即時擷取模式中，LiveCapture 以多執行緒架構監聽網路卡，主執行緒負責封包擷取，背景執行緒定期輸出流量統計儀表板；在離線模式中，PcapAnalyzer 讀取 PCAP 檔案，進行協議統計、HTTP/DNS 提取與 TLS 版本分析。系統另加入 Session 管理機制，每次執行自動建立時間戳記資料夾隔離紀錄，並提供手動初始化指令清空輸出，確保測試環境乾淨。

- 驗證方式：在終端機執行 pytest tests/test_parser.py -v
    - 測試涵蓋各種封包類型與邊界條件

```
(.venv) PS C:\Users\user\Desktop\network_capture> pytest tests/test_parser.py -v
plugins: cov-7.0.0
collected 45 items

tests/test_parser.py::TestTCPPacket::test_basic_fields PASSED                                   [  2%]
tests/test_parser.py::TestTCPPacket::test_syn_ack_flags PASSED                                  [  4%]
tests/test_parser.py::TestTCPPacket::test_psh_ack_flags PASSED                                  [  6%]
tests/test_parser.py::TestTCPPacket::test_http_service_detection PASSED                         [  8%]
tests/test_parser.py::TestTCPPacket::test_ssh_service_detection PASSED                          [ 11%]
tests/test_parser.py::TestTCPPacket::test_https_service_detection PASSED                        [ 13%]
tests/test_parser.py::TestTCPPacket::test_seq_ack_window_fields PASSED                          [ 15%]
tests/test_parser.py::TestTCPPacket::test_ip_version_ipv4 PASSED                                [ 17%]
tests/test_parser.py::TestUDPPacket::test_dns_over_udp PASSED                                   [ 20%]
tests/test_parser.py::TestUDPPacket::test_dhcp_detection PASSED                                 [ 22%]
tests/test_parser.py::TestUDPPacket::test_ntp_detection PASSED                                  [ 24%]
tests/test_parser.py::TestUDPPacket::test_ssdp_detection PASSED                                 [ 26%]
tests/test_parser.py::TestICMPPacket::test_echo_request PASSED                                  [ 28%]
tests/test_parser.py::TestICMPPacket::test_echo_reply PASSED                                    [ 31%]
tests/test_parser.py::TestICMPPacket::test_destination_unreachable_port PASSED                  [ 33%]
tests/test_parser.py::TestICMPPacket::test_destination_unreachable_host PASSED                  [ 35%]
tests/test_parser.py::TestICMPPacket::test_time_exceeded_ttl PASSED                             [ 37%]
tests/test_parser.py::TestICMPPacket::test_time_exceeded_fragment PASSED                        [ 40%]
```

攻擊特徵偵測：

AnomalyDetector 是獨立的偵測引擎，可同時被即時擷取與離線分析兩種模式共用。它針對六種攻擊維護各自的計數器：SYNFlood 計算單一 IP 的純 SYN 封包數、Port Scan 追蹤不重複的目標 Port 數、ARP Spoofing 偵測同一 IP 出現第二個 MAC 位址。系統設計了防重複告警機制，確保同一攻擊只通報一次，避免重複訊息干擾使用者。

- 驗證方式：在終端機執行 pytest tests/test_anomaly.py -v
    - 測試涵蓋各種封包類型與邊界條件
    - 測試範例：驗證 ARP Spoofing 的觸發條件：同 MAC 不觸發，第二個不同 MAC 才觸發。

test_anomaly.py 部分程式

```python
class TestArpSpoofing:

    def test_arp_spoof_triggered_on_second_mac(self, detector, parser):
        """同一 IP 出現第二個 MAC -> 觸發 ARP Spoofing 警告"""
        triggered = []
        detector.on_alert = lambda a: triggered.append(a)

        # 正常 ARP：IP=192.168.1.100, MAC=aa:bb:cc
        pkt1 = (Ether(src="aa:bb:cc:dd:ee:ff")
                / ARP(op=1, psrc="192.168.1.100", pdst="192.168.1.1",
                      hwsrc="aa:bb:cc:dd:ee:ff"))
        r1 = parser.parse(pkt1)
```

```python
        detector.inspect(pkt1, r1)
        assert len(triggered) == 0

        # 欺騙 ARP：同一 IP，不同 MAC
        pkt2 = (Ether(src="11:22:33:44:55:66")
                / ARP(op=2, psrc="192.168.1.100", pdst="192.168.1.1",
                      hwsrc="11:22:33:44:55:66"))
        r2 = parser.parse(pkt2)
        detector.inspect(pkt2, r2)

        spoof_alerts = [a for a in triggered if "ARP" in a["attack_type"]]
        assert len(spoof_alerts) == 1
        assert spoof_alerts[0]["severity"] == "CRITICAL"
        assert "192.168.1.100" in spoof_alerts[0]["src_ip"]

    def test_same_mac_no_alert(self, detector, parser):
        """同一 IP 同一 MAC 多次 -> 不觸發警告"""
        for _ in range(5):
            pkt = (Ether(src="aa:bb:cc:dd:ee:ff")
                   / ARP(op=1, psrc="192.168.1.200", pdst="192.168.1.1",
                         hwsrc="aa:bb:cc:dd:ee:ff"))
            r = parser.parse(pkt)
            detector.inspect(pkt, r)

        assert len(detector.alert_history) == 0
```

輸出結果：

```
(.venv) PS C:\Users\user\Desktop\network_capture> pytest tests/test_anomaly.py -v
============================================================ test session starts ============================================================
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0 -- c:\Users\user\Desktop\network_capture\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\Desktop\network_capture
plugins: cov-7.0.0
collected 17 items

tests/test_anomaly.py::TestSynFlood::test_syn_flood_triggered PASSED                                                                [  5%]
tests/test_anomaly.py::TestSynFlood::test_syn_ack_not_counted PASSED                                                                [ 11%]
tests/test_anomaly.py::TestSynFlood::test_no_duplicate_alert PASSED                                                                 [ 17%]
tests/test_anomaly.py::TestSynFlood::test_different_ips_separate_alerts PASSED                                                      [ 23%]
tests/test_anomaly.py::TestPortScan::test_port_scan_triggered PASSED                                                                [ 29%]
tests/test_anomaly.py::TestPortScan::test_port_scan_details_contain_ports PASSED                                                    [ 35%]
tests/test_anomaly.py::TestPortScan::test_null_scan_classification PASSED                                                           [ 41%]
tests/test_anomaly.py::TestIcmpFlood::test_icmp_flood_triggered PASSED                                                              [ 47%]
tests/test_anomaly.py::TestIcmpFlood::test_icmp_echo_reply_not_counted PASSED                                                       [ 52%]
tests/test_anomaly.py::TestUdpFlood::test_udp_flood_triggered PASSED                                                                [ 58%]
tests/test_anomaly.py::TestArpSpoofing::test_arp_spoof_triggered_on_second_mac PASSED                                               [ 64%]
tests/test_anomaly.py::TestArpSpoofing::test_same_mac_no_alert PASSED                                                               [ 70%]
tests/test_anomaly.py::TestDnsAmplification::test_dns_amp_triggered_on_high_ratio PASSED                                           [ 76%]
tests/test_anomaly.py::TestDnsAmplification::test_normal_dns_no_alert PASSED                                                        [ 82%]
tests/test_anomaly.py::TestDetectorUtility::test_reset_clears_counters PASSED                                                       [ 88%]
tests/test_anomaly.py::TestDetectorUtility::test_get_summary_structure PASSED                                                       [ 94%]
tests/test_anomaly.py::TestDetectorUtility::test_alert_severity_levels PASSED                                                       [100%]

============================================================= 17 passed in 0.52s ==============================================================
(.venv) PS C:\Users\user\Desktop\network_capture>
```

封包特徵影像化：

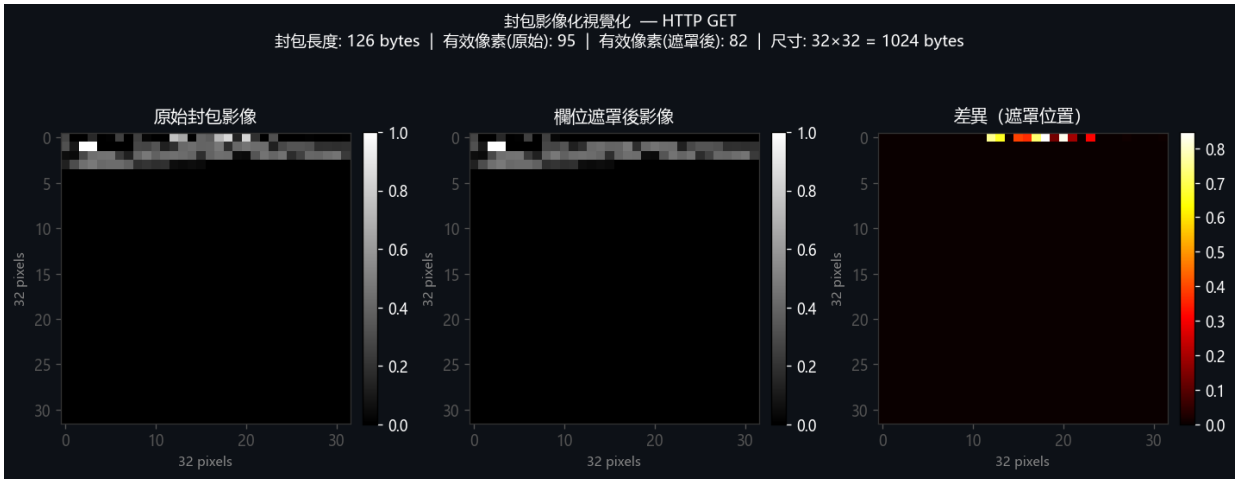
PacketVisualizer 是連接封包與深度學習的橋梁，核心邏輯是將原始封包位元組轉換為 32×32 的 float32 灰階矩陣。轉換過程先遮蔽 IP 位址、Port 與 TCP 序號等識別型欄位，讓模型學習封包的行為模式而非通訊身份，再透過截斷或補零統一長度，最後重塑(reshape)並正規化。DatasetBuilder 批次處理 PCAP 檔案，輸出訓練用的 NumPy 矩陣。

- 驗證方式：在終端機執行 python tests/test_visualization.py
    - 測試範例：生成對比圖

test_visualization.py 部分程式

```
# ── 測試 5：生成對比圖 ──────────────────────────────────────
print("\n[測試 5] 生成對比圖（原始 vs 遮罩後）")
vis = PacketVisualizer("medium", apply_mask=True)
for name, pkt_bytes in packets.items():
    safe_name = name.replace(" ", "_").lower()
    fig = vis.visualize_comparison(
        pkt_bytes,
        label=name,
        save_path=f"{OUTPUT}/compare_{safe_name}.png"
    )
    plt.close(fig)
    print(f"{name}")
```

輸出結果：



Convolutional Autoencoder (CAE) 模型建置與訓練：

CAE 由 CNN、 Encoder 和 Decoder 組成， Encoder 透過四層卷積將 32×32 影像壓縮至 32 維潛在向量，Decoder 以轉置卷積還原為原始尺寸。訓練方式採非監督式學習，只輸入正常流量封包讓模型學習正常封包的重建方式，以均方誤差（Mean Squared Error, MSE）損失最小化輸入與重建影像的差異。訓練完成後根據正常資料重建誤差並設定閾值，超過閾值即判定為異常。並在訓練過程中修正了原始模型因參數量過大而過度泛化的問題。

- 驗證方式一：在終端機執行 pytest tests/test_cnn_model.py -v
    - 測試範例：驗證模型的前向傳播維度正確，以及梯度能正常反向傳播

test_cnn_model.py 部分程式

```
class TestCNNAutoencoder:

    def test_forward_output_shapes(self, model, sample_batch):
        """forward() 應回傳正確 shape 的 (x_hat, z)"""
        x_hat, z = model(sample_batch)
        assert x_hat.shape == sample_batch.shape, \
            f"x_hat shape {x_hat.shape} ≠ input shape {sample_batch.shape}"
        assert z.shape == (4, 64)

    def test_reconstruction_error_shape(self, model, sample_batch):
        """reconstruction_error() 應回傳 shape=(batch,) 的 1D tensor"""
        errors = model.reconstruction_error(sample_batch)
        assert errors.shape == (4,), f"期望 (4,)，得到 {errors.shape}"

    def test_reconstruction_error_non_negative(self, model, sample_batch):
        """重建誤差（MSE）應為非負數"""
        errors = model.reconstruction_error(sample_batch)
        assert (errors >= 0).all(), "重建誤差不應為負數"

    def test_identical_input_low_error(self, model):
        """若輸入接近 Decoder 的輸出分布，誤差應較低"""
        # 建立接近中間值（0.5）的輸入：Sigmoid 輸出傾向集中在中間
        x_mid = torch.ones(2, 1, 32, 32) * 0.5
        x_rand = torch.rand(2, 1, 32, 32)
        # 兩者誤差都應有限（smoke test，不比大小）
        e_mid  = model.reconstruction_error(x_mid)
        e_rand = model.reconstruction_error(x_rand)
        assert e_mid.max().item() < 10.0   # MSE 不應爆炸

    def test_encode_decode_methods(self, model, sample_batch):
        """encode() 與 decode() 的單獨呼叫應與 forward() 一致"""
        z     = model.encode(sample_batch)
        x_hat = model.decode(z)
        assert x_hat.shape == sample_batch.shape

    def test_model_info(self, model):
        """get_model_info() 應回傳合理的模型資訊"""
        info = model.get_model_info()
```

```python
        assert info["latent_dim"] == 64
        assert info["total_params"] > 0
        assert info["trainable_params"] == info["total_params"]  # 所有參數可訓練
        assert info["model_size_MB"] > 0
        print(f"\n  模型參數: {info['total_params']:,}，"
              f"大小: {info['model_size_MB']:.2f} MB")

    def test_gradient_flow(self, small_model, sample_batch):
        """訓練梯度應能從 Loss 反向傳播到所有參數"""
        optimizer = torch.optim.Adam(small_model.parameters())
        x_hat, _ = small_model(sample_batch)
        loss = torch.nn.MSELoss()(x_hat, sample_batch)
        optimizer.zero_grad()
        loss.backward()

        # 確認所有可訓練參數都有梯度
        for name, param in small_model.named_parameters():
            if param.requires_grad:
                assert param.grad is not None, f"{name} 沒有梯度"
                assert not torch.isnan(param.grad).any(), f"{name} 梯度為 NaN"
```

輸出結果:

```
問題    輸出    偵錯主控台    終端機    連接埠                                    powershell

(.venv) PS C:\Users\user\Desktop\network_capture> pytest tests/test_cnn_model.py -v
plugins: cov-7.0.0
collected 22 items

tests/test_cnn_model.py::TestEncoder::test_output_shape PASSED                                  [  4%]
tests/test_cnn_model.py::TestEncoder::test_custom_latent_dim PASSED                             [  9%]
tests/test_cnn_model.py::TestEncoder::test_output_requires_grad PASSED                          [ 13%]
tests/test_cnn_model.py::TestEncoder::test_different_batch_sizes PASSED                         [ 18%]
tests/test_cnn_model.py::TestDecoder::test_output_shape PASSED                                  [ 22%]
tests/test_cnn_model.py::TestDecoder::test_output_range PASSED                                  [ 27%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_forward_output_shapes PASSED                  [ 31%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_reconstruction_error_shape PASSED             [ 36%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_reconstruction_error_non_negative PASSED      [ 40%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_identical_input_low_error PASSED              [ 45%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_encode_decode_methods PASSED                  [ 50%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_model_info PASSED                             [ 54%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_gradient_flow PASSED                          [ 59%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_different_latent_dims PASSED                  [ 63%]
tests/test_cnn_model.py::TestTrainerSmoke::test_single_batch_training PASSED                    [ 68%]
tests/test_cnn_model.py::TestTrainerSmoke::test_loss_decreases_over_iterations PASSED           [ 72%]
tests/test_cnn_model.py::TestTrainerSmoke::test_reconstruction_improves_after_training PASSED   [ 77%]
tests/test_cnn_model.py::TestPacketDataset::test_from_numpy PASSED                              [ 81%]
tests/test_cnn_model.py::TestPacketDataset::test_dataloader_batch PASSED                        [ 86%]
tests/test_cnn_model.py::TestAnomalyScorer::test_score_returns_tuple PASSED                     [ 90%]
tests/test_cnn_model.py::TestAnomalyScorer::test_anomaly_threshold_logic PASSED                 [ 95%]
tests/test_cnn_model.py::TestAnomalyScorer::test_score_batch_length PASSED                      [100%]
```

- 驗證方式二：在終端機執行 python run_training.py --epochs 100 --latent 32
    - 測試範例：透過完整訓練流程驗證從資料載入到模型評估的所有模組串接正確

```
tests/test_cnn_model.py::TestCNNAutoencoder::test_gradient_flow PASSED                                  [ 59%]
tests/test_cnn_model.py::TestCNNAutoencoder::test_different_latent_dims PASSED                          [ 63%]
tests/test_cnn_model.py::TestTrainerSmoke::test_single_batch_training PASSED                            [ 68%]
tests/test_cnn_model.py::TestTrainerSmoke::test_loss_decreases_over_iterations PASSED                   [ 72%]
...
  ==================================================
  指標                                  值
  ──────────────────────────────────────
  Precision                    0.9231
  Recall（偵測率）                  1.0000
  F1 Score                     0.9600
  Accuracy                     0.9688
  FPR（誤報率）                     0.0500
  AUC-ROC                      1.0000
  ──────────────────────────────────────
  TP=300  TN=475  FP=25  FN=0
  ==================================================
  [圖表] 混淆矩陣已儲存: output/model\confusion_matrix.png

  正常流量平均誤差: 0.000681
  攻擊流量平均誤差: 0.001926
  兩者誤差比值     : 2.83x
  [圖表] ROC/PR 曲線已儲存: output/model\roc_pr_curve.png
  [圖表] 分數分布圖已儲存: output/model\score_distribution.png

============================================================
  完成！輸出目錄: C:\Users\user\Desktop\network_capture\output\model
    best_model.pt                        1425.7 KB
    confusion_matrix.png                 24.8 KB
    error_distribution.png               29.6 KB
    reconstruction_samples.png           38.8 KB
    roc_pr_curve.png                     76.0 KB
    score_distribution.png               47.2 KB
    training_curve.png                   39.8 KB
    training_result.json                 5.8 KB
============================================================
```

資料集整合：

DatasetFactory 提供統一的資料集載入介面，底層分別由三個載入器 (Loader) 處理各資料集的格式差異。NSL-KDD 有三個類別型特徵需要 LabelEncoder(標籤編碼器) 編碼；CIC-IDS2017 的標籤欄位名稱含前置空格，且 CICFlowMeter 輸出常含大量無限值需要清理；CIC-DDoS2019 的 CSV 分散於子目錄，需要遞迴掃描。各 Loader 完成前處理後，統一輸出 32×32 影像矩陣儲存為 .npy (NumPy 函式庫專屬的二進位陣列儲存格式) 檔案供訓練器(Trainer) 使用。

- 驗證方式：在終端機執行 pytest tests/test_dataset_loader.py -v
    - 測試範例：測試使用模擬的小型 CSV(不使用真實資料集)，驗證無限值處理與 CIC 標籤前方的空格(格式問題)

test_dataset_loader.py 部分程式

```
    def test_infinite_value_handling(self, tmp_dir):
        """含有 inf / -inf 的資料應被正確處理（不出現在輸出中）"""
        feature_names = [f"F{i}" for i in range(10)]
        data = np.random.rand(10, 10).astype(np.float32)
        # 插入無限值
        data[0, 0] = np.inf
        data[1, 1] = -np.inf
        data[2, 2] = np.nan

        df = pd.DataFrame(data, columns=feature_names)
        df[" Label"] = ["BENIGN"] * 5 + ["Attack"] * 5
        path = os.path.join(tmp_dir, "test_inf.csv")
```

```python
        df.to_csv(path, index=False)

        loader = CICIDSLoader(tmp_dir)
        X_normal, X_attack, _ = loader.load(csv_files=[path])

        assert not np.isnan(X_normal).any()
        assert not np.isinf(X_normal).any()

    def test_file_not_found_raises(self, tmp_dir):
        """目錄為空時應拋出 FileNotFoundError"""
        empty_dir = os.path.join(tmp_dir, "empty")
        os.makedirs(empty_dir)
        loader = CICIDSLoader(empty_dir)
        with pytest.raises(FileNotFoundError):
            loader.load()

    def test_max_sample_limit(self, cicids_dir):
        """max_normal / max_attack 參數應正確限制樣本數"""
        loader = CICIDSLoader(cicids_dir)
        X_normal, X_attack, _ = loader.load(max_normal=10, max_attack=5)

        assert len(X_normal) <= 10
        assert len(X_attack) <= 5

    def test_label_vector_length(self, cicids_dir):
        """y 的長度應等於 X_normal + X_attack"""
        loader = CICIDSLoader(cicids_dir)
        X_normal, X_attack, y = loader.load()
        assert len(y) == len(X_normal) + len(X_attack)
```

```
(.venv) PS C:\Users\user\Desktop\network_capture> pytest tests/test_dataset_loader.py -v
========================================================== test session starts ==========================================================
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0 -- c:\Users\user\Desktop\network_capture\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\Desktop\network_capture
plugins: cov-7.0.0
collected 37 items

tests/test_dataset_loader.py::TestFeaturesToImages::test_output_shape_41_features PASSED                                   [  2%]
tests/test_dataset_loader.py::TestFeaturesToImages::test_output_shape_78_features PASSED                                   [  5%]
tests/test_dataset_loader.py::TestFeaturesToImages::test_padding_correctness PASSED                                        [  8%]
tests/test_dataset_loader.py::TestFeaturesToImages::test_truncation_over_1024 PASSED                                       [ 10%]
tests/test_dataset_loader.py::TestFeaturesToImages::test_output_dtype PASSED                                               [ 13%]
tests/test_dataset_loader.py::TestFeaturesToImages::test_single_sample PASSED                                              [ 16%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_load_returns_correct_shapes PASSED                                    [ 18%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_load_correct_counts PASSED                                            [ 21%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_label_vector_consistency PASSED                                       [ 24%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_output_value_range PASSED                                             [ 27%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_output_dtype PASSED                                                   [ 29%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_file_not_found_raises PASSED                                          [ 32%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_categorical_encoding PASSED                                           [ 35%]
tests/test_dataset_loader.py::TestNSLKDDLoader::test_y_dtype PASSED                                                        [ 37%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_load_returns_correct_shapes PASSED                                    [ 40%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_load_correct_counts PASSED                                            [ 43%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_label_column_with_leading_space PASSED                                [ 45%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_value_range PASSED                                                    [ 48%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_no_nan_in_output PASSED                                               [ 51%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_infinite_value_handling PASSED                                        [ 54%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_file_not_found_raises PASSED                                          [ 56%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_max_sample_limit PASSED                                               [ 59%]
tests/test_dataset_loader.py::TestCICIDSLoader::test_label_vector_length PASSED                                            [ 62%]
tests/test_dataset_loader.py::TestCICDDoS2019Loader::test_recursive_csv_scan PASSED                                        [ 64%]
tests/test_dataset_loader.py::TestCICDDoS2019Loader::test_output_shape PASSED                                              [ 67%]
tests/test_dataset_loader.py::TestCICDDoS2019Loader::test_inherits_cicids_preprocessing PASSED                             [ 70%]
tests/test_dataset_loader.py::TestDatasetFactory::test_simulate_returns_correct_shapes PASSED                              [ 72%]
tests/test_dataset_loader.py::TestDatasetFactory::test_simulate_label_counts PASSED                                        [ 75%]
tests/test_dataset_loader.py::TestDatasetFactory::test_simulate_value_range PASSED                                         [ 78%]
tests/test_dataset_loader.py::TestDatasetFactory::test_simulate_dtype PASSED                                               [ 81%]
tests/test_dataset_loader.py::TestDatasetFactory::test_simulate_no_nan PASSED                                              [ 83%]
tests/test_dataset_loader.py::TestDatasetFactory::test_invalid_dataset_name_raises PASSED                                  [ 86%]
tests/test_dataset_loader.py::TestDatasetFactory::test_case_insensitive_name PASSED                                        [ 89%]
```

二、目前約完成 25 %之進度

比預計的進度（■稍微□嚴重）（□落後■超前）。

主要原因：

封包的擷取與解析、影像化、CNN 模型的相關程式實作已經完成，但模型仍需要訓練與調整

三、下次主要工作

(進度報告 2) 模型訓練與重建誤差閾值(Threshold)調校

下一步的重點工作是使用 CIC-IDS2017 真實資料集重新訓練模型，以真實的正常流量與 DDoS、Port Scanning（端口掃描） 攻擊流量評估模型的實際偵測能力，並針對不同攻擊類型調整閾值百分位數，找出精確率（Precision） 與 召回率（Recall）的最佳平衡點。同時規劃三組消融實驗，分別驗證欄位遮罩、影像尺寸與損失函數的設計決策對偵測效能的貢獻，為最終模型的設計選擇提供實驗依據。

若時間充足，會研究關於 ai agent 與 XAI 的相關資料，使用 n8n (暫定) 開源工具去做模性可解釋性的實作。

四、遭遇問題(實作、技術或溝通問題等)

請描述遭遇問題/目前解決進度/解決方法/未解決原因：

<u>問題 1 : 封包的擷取與解析以及 CNN 的實作功能難以確認是否能正常運作。</u>

解決方法 : 進行單元測試，驗證程式功能是否能正常運作，包含邊界測試。

<u>問題 2：CNN 模型評估失敗(TP=0，零偵測率)</u>

問題描述：使用 150 筆正常封包訓練後，評估結果 TP=0、FN=160，所有攻擊封包均未被偵測到，Recall = 0、F1 score = 0。推測是因為模型參數量(4.8M)遠大於訓練資料規模(150 筆)，導致過度泛化。攻擊封包的重建誤差與正常封包幾乎相同，導致閾值無法區分兩者。

解決方式：縮小模型並增加訓練資料，加入多樣化 HTTP/DNS/SSH Payload 提高正常流量的識別特徵。

<u>問題 3：UDP 協議誤判(NTP/SSDP 被識別為 DNS)</u>

問題描述：Scapy 建立 UDP 封包時，來源 Port 預設是 53 (DNS Port)，程式只看來源 Port 是否為 53 來判斷 DNS，導致未明確指定 sport 的 NTP/SSDP 封包因預設值被誤判。

解決方式： dport=53 才直接判定為 DNS 查詢；sport=53 需同時具有 DNS layer (pkt.haslayer(DNS))才判定為 DNS 回應

<u>問題 4：PCAP Global Header 格式錯誤，讀取封包數偏少</u>

問題描述：test_visualization.py 測試 10 (純 Python PCAP 讀取)失敗：預期讀取 6 個封包，實際只讀到 5 個。原因是 PCAP Global Header 標準規格為 24 bytes，但程式碼使用的格式字串(6 個欄位)只寫出 20 bytes，缺少有效數字(sig figs) 欄位。讀取端嘗試跳過 24 bytes 的 Global Header 時，多吃了第一個封包的前 4bytes，造成後續封包位置偏移，最終少讀一個封包。

解決方式：格式字串改為 7 個欄位，補上有效數字欄位(sig figs = 0) ，確保 Global Header 正確寫出 24 bytes。

問題 5：Windows 網路介面名稱顯示為 GUID 路徑，無法辨識
問題描述： 列出網路介面時，名稱全是 \Device\NPF_{一串英數字} 的格式，，無法判斷哪個介面是 Wi-Fi 或有線網路，不知道應輸入哪個名稱進行即時擷取。
解決方式：改用 Scapy 的另一個函式取得人類可讀的名稱(例如：Wi-Fi、乙太網路)再顯示。

問題 6：CIC 資料集的欄位名稱有多餘空格
問題描述：讀取 CIC-IDS2017 資料集時，找不到標籤欄位，正常流量和攻擊流量無法分開。原因是 CICFlowMeter 輸出的欄位名稱是 " Label"(前面有一個空格)，程式碼直接比對字串時因為空格而找不到欄位。
解決方式：在比對前先去除空格(.strip())，同時相容 "Label"、" Label"、"label" 三種寫法。

五、老師建議：
(包含學長姊的建議)

- 為甚麼選擇 Convolutional Autoencoder (CAE,捲積自編碼器)
- 對 UI/UX 的規劃以及開發工具？建議能讓沒有資訊背景的也能清楚如何進行操作
- 模性可解釋性是什麼? 要如何實現可解釋性(XAI)?

六、回覆建議：

- 為甚麼選擇 Convolutional Autoencoder ?
    - 網路攻擊偵測最大的挑戰是攻擊樣本難以事先收集，無法像一般分類問題一樣準備大量帶標籤的攻擊資料。CAE 採用非監督式學習，訓練時只需要正常流量，讓模型學習正常封包長什麼樣子，遇到從未見過的攻擊樣式時，重建誤差自然偏高，因此能偵測零時差攻擊(Zero-day Attack)。選擇卷積架構而非一般全連接 Autoencoder，是因為封包被轉換成 32×32 的影像格式，卷積層能有效捕捉封包位元組之間的空間分布模式(例如 TCP Header 的固定結構、Payload 的資料分布)，比單純把所有像素攤平處理效果更好。

- UI/UX 的規劃以及開發的工具
    - 操作介面的設計重點是讓沒有程式背景的人也能清楚操作。因此介面以儀表板為主，避免使用者直接面對命令列，所有功能透過視覺化按鈕與圖表呈現
    - 頁面規劃
        - 即時監控頁面：顯示目前網路流量統計、協議分布圓餅圖、每秒封包數折線圖，以及即時告警通知（紅色提示框）。使用者可以在這個頁面選擇要監聽的網路介面並啟動擷取
        - PCAP 分析頁面：上傳 PCAP 檔案後，自動產生分析報告，包含封包影像熱力圖、攻擊偵測結果，以及 Grad-CAM 視覺化說明
        - AI Agent 對話頁面：嵌入 n8n 串接的聊天介面，使用者可以詢問資安問題或請 AI 說明目前的分析結果
        - 歷史紀錄頁面：設置管理員系統，列出資料庫所有過去的分析紀錄，可以篩選、查看與下載報告

- 開發的工具 : 使用 Figma、penpot 等免費 UI 設計工具進行設計，再移植到 Django
- 參考這二本書進行設計
  - Refactoring UI (author : Adam Wathan & Steve Schoger)
  - Information Dashboard Design: The Effective Visual Commmunication of Data (author : Stephen Few)

- XAI 可解釋性的實現計畫
  1. Grad-CAM 視覺化:
     將 CNN 模型對重建誤差的梯度轉換為熱力圖，疊加在封包影像上，標示出封包中哪個位元組區域導致異常判定。因此可根據熱力圖的高亮區域，協助判斷異常來源
  2. AI Agent (n8n + 免費 API key)
     Grad-CAM 的熱力圖對一般使用者仍然抽象，因此計畫在前端整合一個 AI Agent，將技術結果轉換為自然語言說明。使用 n8n 作為工作流程串接平台，搭配免費 API (Google Gemini API 免費方案)，目前規劃以下三個核心功能：
     - **科普資安**：使用者可以直接進行詢問，例如:什麼是 SQL injection？、DDOS 會造成什麼危害？，Ai Agent 以簡單語言回覆，不需要資安背景也能理解。
     - **平台操作指引**：使用者輸入想做的事情 ，例如 : 我想分析這個 PCAP 檔案，Ai Agent 回傳對應的操作步驟與指令，引導使用者正確使用平台功能。
     - **報告分析結果說明**：平台產生告警或評估報告後，Ai Agent 自動解讀數值，例如: Recall=1.00、F1 score =0.5 代表什麼意思，並以一段文字摘要說明偵測結果。